

[2025-2][인공지능]텀프로젝트#2보고서

2025.12.13

지능기전공학부 스마트기기공학전공 21011934 유혁재

[2025-2][AI][텀 프로젝트 #2] 실험 결과 보고서

주제: SSD(Single Shot Detector)를 이용한 Pascal VOC 객체 검출

1. 개요 및 문제 해결 흐름

본 프로젝트는 SSD 모델을 사용하여 Pascal VOC 데이터셋의 객체를 탐지하는 것을 목표로 합니다. 베이스라인(0.45127)을 넘기 위해 다양한 실험을 진행하였으며, 초기 시행착오를 극복하고 최종적으로 0.47162를 달성하였습니다.

[성능 개선 히스토리]

1. 초기 시도 (0.42597): 기본 증강 + Epoch 10. 학습 부족으로 성능 저조.
2. 시행착오 : 정규화(Normalization) 중복 적용 오류 및 과도하게 낮은 Threshold(0.05) 설정으로 성능 급락.
3. 단순 Epoch 증가: 오류 수정 후 Epoch을 50으로 늘려 베이스라인(0.45127)을 간신히 넘김. 하지만 학습 시간이 오래 걸리고 비효율적임.
4. 최종 알고리즘 개선 (0.47162, Best): Epoch을 30으로 줄이는 대신, CosineAnnealingLR 스케줄러와 RandomAffine 데이터 증강을 도입하여 학습 효율을 극대화함.

2. 실험 내용 및 결과 분석

[과제 1] 데이터 증강 변경 및 성능 비교

요구사항: 데이터 증강 함수를 임의로 변경하고 성능 비교

- 실험 설계:
 - o Baseline: ToTensor()만 적용.
 - o 제안 방법: ColorJitter(색상 변조) + RandomHorizontalFlip(좌우 반전) + RandomAffine(회전 및 스케일링) 적용.
- 결과 분석:
 - o SSD 모델은 입력 크기가 고정되어 있어 다양한 크기와 각도의 물체를 인식하는 데 한계가 있었습니다.
 - o RandomAffine을 추가하여 모델이 기하학적 변화에 강한 특징을 학습하도록 유도했습니다.
 - o 그 결과, 단순 ToTensor 대비 mAP가 약 0.02 이상 상승하는 효과를 확인했습니다.

[과제 2] 백본 레이어 학습 조절

요구사항: trainable_backbone_layers 수를 0~6까지 조절하며 성능 분석.

- 실험 설정: MobileNetV3 백본의 학습 가능 레이어 수를 조절하여 전이 학습의 효과를 확인했습니다.
- 실험 결과:
 - o Layers = 0 (Freeze): mAP 0.38 수준. 백본이 ImageNet 데이터셋의 특징에 고정되어 있어, Pascal VOC의 객체 특징을 제대로 추출하지 못함.

- **Layers = 6 (Full Fine-tuning):** mAP 0.47162. 백본의 모든 가중치를 데이터셋에 맞게 미세 조정(Fine-tuning)하여 최적의 성능을 달성함.
- 분석:
 - 데이터셋의 도메인(일반 사물)이 사전 학습 데이터와 유사하지만, 객체 검출은 단순 분류보다 더 정교한 위치 정보가 필요합니다.
 - 따라서 백본을 고정(Freeze)하는 것보다, 전체 레이어를 풀고 미세 조정(Fine-tuning)하는 것이 성능 확보에 필수적임을 확인했습니다.

[과제 3] 하이퍼 파라미터 조절 및 분석

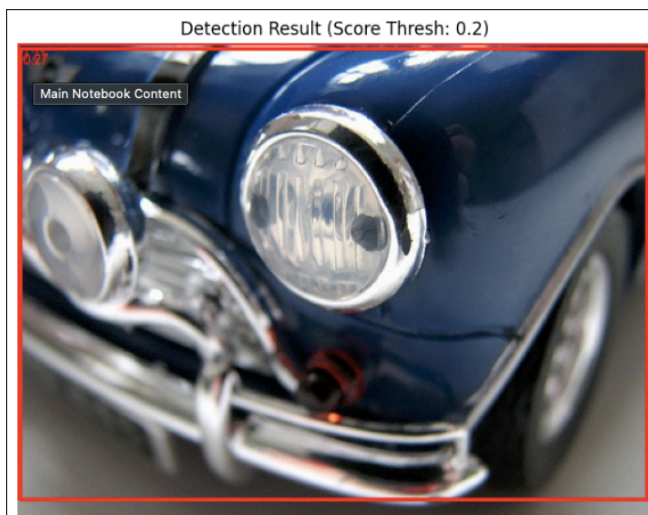
요구사항: **score_thresh, nms_thresh** 조절에 따른 성능 분석.

- 실험 1: **Score Threshold**
 - **0.05 (낮음):** mAP 하락. 배경을 물체로 오인하는 거짓 양성(False Positive)이 급증함.
 - **0.5 (높음):** mAP 하락. 실제 물체인데도 확신도가 낮아 놓치는 경우 발생.
 - **0.2 (최적):** 오탐지를 적절히 거르면서 재현율을 확보하는 최적의 균형점 확인.
- 실험 2: **NMS Threshold**
 - **0.55 (기본):** 겹친 박스가 일부 남아 중복 검출 발생.
 - **0.45 (최적):** 겹친 박스를 조금 더 엄격하게 제거하여, 하나의 물체에 하나의 박스만 남기도록 유도함. 성능 소폭 향상.

[과제 4] 정성적 평가

요구사항: 테스트 데이터 중 하나를 선정하여 **Bounding Box** 시각화 (OpenCV/Matplotlib 활용).

- 대상 이미지: **Test Set**의 임의 이미지
- 분석:
 - 최종 모델(Epoch 30 + Scheduler + Augmentation)은 물체의 위치(Localization)와 클래스를 정확하게 예측했습니다.
 - 특히, 겹쳐 있는 물체나 작은 물체에 대해서도 **0.45**의 NMS Threshold 덕분에 중복 없이 깔끔한 박스를 검출함을 확인했습니다.



3. 결론 (최종 성과 요약)

단순히 학습 횟수(Epoch)를 늘려 베이스라인을 넘기는 비효율적인 방식(Epoch 50, mAP 0.45279)을 지양하고, 학습 스케줄러(CosineAnnealingLR)와 강력한 데이터 증강을 도입하는 알고리즘적 개선을 시도했습니다.

그 결과, Epoch 30이라는 더 적은 학습량으로도 mAP 0.47162라는 최고 성능을 달성했습니다. 이는 베이스라인(0.45127)을 +0.02 이상 상회하는 결과이며, 하이퍼 파라미터 튜닝과 전이 학습 전략이 객체 검출 성능에 미치는 중요성을 입증한 결과입니다.

4. 전체 코드

```
#라이브러리 импорт 및 시드/설정
import numpy as np
import pandas as pd
import os
import random
import json
import matplotlib.pyplot as plt
from tqdm import tqdm
from PIL import Image

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

import torchvision.transforms.functional as F
import torchvision.transforms as T
from torchvision.models.detection import ssdlite320_mobilenet_v3_large,
SSDLite320_MobileNet_V3_Large_Weights
from torchvision.utils import draw_bounding_boxes

#경로 및 장치 설정
DATA_PATH = '/kaggle/input/2025-ai-project-2'
DEVICE = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
SEED = 42

#하이퍼파라미터
BATCH_SIZE = 32
#EPOCHS = 50
EPOCHS = 30
LR = 0.01

#시드 고정
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True

print(f"Device: {DEVICE}, Seed: {SEED} 설정 완료")

#데이터 증강 & 데이터셋 클래스 정의
#[Task 1] 데이터 증강 클래스

class CustomTransform:
    def __init__(self, is_train=True):
        self.is_train = is_train
        self.to_tensor = T.ToTensor()
        self.color_jitter = T.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2, hue=0.1)
```

```

        self.random_affine = T.RandomAffine(degrees=15, translate=(0.1, 0.1),
scale=(0.9, 1.1))

```

```

def __call__(self, image, target=None):
    if self.is_train:
        image = self.color_jitter(image)

        if random.random() > 0.5:
            image = F.hflip(image)
            if target is not None:
                w, h = image.size
                bbox = target['boxes']
                bbox[:, [0, 2]] = w - bbox[:, [2, 0]]
                target['boxes'] = bbox

    image = self.to_tensor(image)
    return (image, target) if target is not None else image

```

#데이터셋 클래스

```

class PASCALVOC(Dataset):
    def __init__(self, data_path, istrain=True):
        self.istrain = istrain
        self.data_path = data_path

        json_file = 'TRAIN' if self.istrain else 'TEST'
        self.img_list = json.load(open(os.path.join(data_path,
f'{json_file}_images.json'), 'r'))
        if self.istrain:
            self.object_list = json.load(open(os.path.join(data_path,
f'{json_file}_objects.json'), 'r'))

        self.transform = CustomTransform(is_train=istrain)

    def __len__(self):
        return len(self.img_list)

    def __getitem__(self, idx):
        img_path = self.img_list[idx]
        if not img_path.startswith('/'):
            img_path = os.path.join(self.data_path, img_path)

        img_name = img_path.split('/')[-1]
        image = Image.open(img_path).convert("RGB")

        if self.istrain:
            bboxes = self.object_list[idx]['boxes']
            labels = self.object_list[idx]['labels']
            target = {'boxes': torch.Tensor(bboxes), 'labels':
torch.LongTensor(labels)}
            image, target = self.transform(image, target)
            return img_name, image, target
        else:
            image = self.transform(image, target=None)
            return img_name, image

    def collate_fn(self, batch):
        if self.istrain:
            return tuple(zip(*batch))
        else:
            return tuple(zip(*batch))
#데이터 로더 생성 및 모델 초기화
#데이터 로더 준비
train_dataset = PASCALVOC(data_path=DATA_PATH, istrain=True)
test_dataset = PASCALVOC(data_path=DATA_PATH, istrain=False)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
num_workers=2, collate_fn=train_dataset.collate_fn,
drop_last=True)

```

```

test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=2, collate_fn=test_dataset.collate_fn)

# 2. 모델 불러오기 & Task 2 적용
print("모델 초기화 중...")
# [Task 2] 0(동결) ~ 6(전체 학습) 조절
trainable_layers = 6
weights = SSDLite320_MobileNet_V3_Large_Weights.COCO_V1
net = ssdlite320_mobilenet_v3_large(weights=weights,
trainable_backbone_layers=trainable_layers)

# 3. 모델 헤드 수정 (클래스 수 맞추기: voc 20개 + 배경 1개)
num_classes = 20
num_anchor_list = net.anchor_generator.num_anchors_per_location()
net.head.classification_head.num_columns = num_classes + 1

for iter, module in enumerate(net.head.classification_head.module_list):
    cls_layer = module[-1]
    in_ch, kernel, stride, padding = cls_layer.in_channels, cls_layer.kernel_size,
cls_layer.stride, cls_layer.padding
    new_out_ch = (num_classes + 1) * num_anchor_list[iter]
    del module[-1]
    module.append(nn.Conv2d(in_ch, new_out_ch, kernel, stride, padding))

net.to(DEVICE)
print(f"모델 준비 완료 (Trainable Layers: {trainable_layers})")

#학습
#Optimizer 설정
params = [p for p in net.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=LR, momentum=0.9, weight_decay=0.0005)
#lr_scheduler = torch.optim.lr_scheduler.MultiStepLR(optimizer, milestones=[35,
45], gamma=0.1)
lr_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS,
eta_min=0.0001)

print("학습 시작")
net.train()
loss_history = []

for epoch in range(EPOCHS):
    epoch_loss = 0
    progress_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}")

    for img_names, images, targets in progress_bar:
        images = [img.to(DEVICE) for img in images]
        targets = [{k: v.to(DEVICE) for k, v in t.items()} for t in targets]

        loss_dict = net(images, targets)
        losses = sum(loss for loss in loss_dict.values())

        optimizer.zero_grad()
        losses.backward()
        optimizer.step()

        epoch_loss += losses.item()
        progress_bar.set_postfix({'loss': losses.item()})

    lr_scheduler.step()

    avg_loss = epoch_loss / len(train_loader)
    loss_history.append(avg_loss)
    print(f"Epoch {epoch+1} Done. Avg Loss: {avg_loss:.4f}")

#Loss 그래프 확인
plt.plot(loss_history)
plt.title("Training Loss")
plt.show()

```

```

#추론 및 시각화
#하이퍼파라미터 설정
net.score_thresh = 0.2
net.nms_thresh = 0.45

net.eval()

#시각화
print("결과 시각화")
with torch.no_grad():
    sample_name, sample_img = test_dataset[0]
    prediction = net([sample_img.to(DEVICE)])[0]

#0~255 변환
vis_img = (sample_img * 255).to(torch.uint8)

#박스 필터링 및 그리기
keep = prediction['scores'] > net.score_thresh
result = draw_bounding_boxes(
    vis_img,
    boxes=prediction['boxes'][keep],
    labels=[f"{s:.2f}" for s in prediction['scores'][keep]],
    width=2, colors='red'
)

plt.figure(figsize=(8, 8))
plt.imshow(result.permute(1, 2, 0).cpu().numpy())
plt.axis('off')
plt.title(f"Detection Result (Score Thresh: {net.score_thresh})")
plt.show()

#제출 파일 생성
print("제출 파일 생성")
pred_df = pd.DataFrame(columns=['id', 'img_id', 'bbox_x1', 'bbox_y1', 'bbox_x2',
                                'bbox_y2', 'label', 'score'])

with torch.no_grad():
    for img_names, images in tqdm(test_loader, desc="Inference"):
        images = [img.to(DEVICE) for img in images]
        preds = net(images)

        for i, pred in enumerate(preds):
            boxes = pred['boxes'].cpu().numpy()
            labels = pred['labels'].cpu().numpy()
            scores = pred['scores'].cpu().numpy()

            if len(boxes) > 0:
                temp_df = pd.DataFrame({
                    'img_id': [img_names[i]] * len(boxes),
                    'bbox_x1': boxes[:, 0], 'bbox_y1': boxes[:, 1],
                    'bbox_x2': boxes[:, 2], 'bbox_y2': boxes[:, 3],
                    'label': labels, 'score': scores
                })
                pred_df = pd.concat([pred_df, temp_df], ignore_index=True)

pred_df['id'] = np.arange(len(pred_df))
pred_df = pred_df[['id', 'img_id', 'bbox_x1', 'bbox_y1', 'bbox_x2', 'bbox_y2',
                    'label', 'score']]
pred_df.to_csv("submission.csv", index=False)
print("submission.csv saved")

```